

Neural Networks

Neural networks (NN) were originally developed as an attempt to emulate the human brain. The brain has about 1.5×10^{10} neurons each with 10 to 10^4 connections called synapses. The speed of messages between neurons is about 100 m/sec which is much slower than CPU speed. Given that our fastest reaction time is around 100 ms and neuron computation time is 1–10 ms, then the number of steps must be less than 100. This is inconceivably small for a sequential computation, even in machine code; therefore, the brain must be computing in parallel. The original idea behind neural networks was to use a computer-based model of the human brain to perform complex tasks. We can recognize people in fractions of a second, but this task is difficult for computers. So why not make software more like the human brain? Despite the promise, there are some drawbacks. The brain model of connected neurons, first suggested by McCulloch and Pitts (1943), is too simplistic given more recent research. For these and other reasons, the methodology is more properly called *artificial* neural nets. As with artificial intelligence, the promise of NNs is not matched by the reality of their performance. There was a large amount of hype concerning NNs but that is now past. Nevertheless, viewed simply as an algorithmic procedure, NNs are competitive with other less ambitiously named methods. NNs are used for various purposes. They can be used as biological models, which was the original motivation. They can also be used as a hardware implementation for adaptive control. But the area of application we are interested in is data analysis. There are NN models that rival the regression, classification and clustering methods normally used by statisticians. A perceptron is a model of a neuron and is the basic building block of a neural network as depicted in Figure 17.1. The output x_o is determined from inputs x_i :

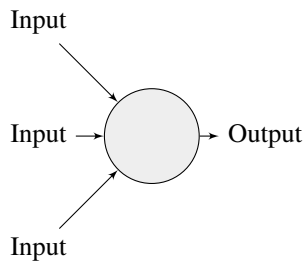


Figure 17.1 A *perceptron*.

$$x_o = f_o\left(\sum_{\text{inputs:}i} w_i x_i\right)$$

where f_o is called the *activation function*. Standard choices include the identity, logistic and indicator functions. The w_i are *weights*. The NN *learns* the weights from the data. A statistician would prefer to say that the NN estimates the parameters from the data. Thus NN terminology differs from statistical usage in ways that can be confusing.

17.1 Statistical Models as NNs

Three common statistical models are analogous to the single perceptron NN. For multiple linear regression:

$$y = \sum_i w_i x_i$$

So here f_o is the identity function. We can define $x_1 \equiv 1$ to get an intercept term. The NN alternative is to attach a weight, called a *bias*, to each neuron:

$$f(x) = x + \theta$$

A statistician would call the bias θ an intercept.

Logistic regression also fits easily within this framework if we define f_o as the logistic function. Of course, such an NN is not exactly equivalent to the corresponding statistical model unless that NN is fit in a very particular way.

Linear discriminant analysis is used to classify a binary response. Suppose there are two groups encoded by $y = 0$ or $y = 1$. In this case f_o is the indicator function. Again the NN and statistical approach are not exactly equivalent unless the same fitting procedures are used.

Other common statistical models can be approximated by adding more neurons. Multivariate multiple linear regression with a bivariate response is $Y = X\beta + \varepsilon$ where Y, X, β, ε are all matrices and is depicted as an NN in Figure 17.2: polynomial regression can be mimicked by using a different activation function and more than one layer of neurons as seen in the second panel of Figure 17.2.

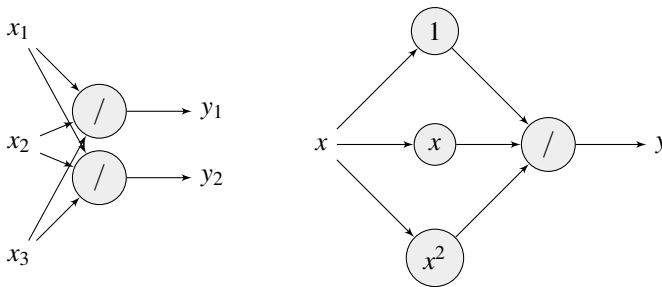


Figure 17.2 NN equivalent of multivariate linear regression is shown on the left. It uses linear activation functions. Polynomial regression is shown on the right and uses powers for activation functions.

17.2 Feed-Forward Neural Network with One Hidden Layer

The feed-forward neural network with one hidden layer is the most common choice for regression-like modeling applications. It takes the form:

$$y_o = \phi_o\left(\sum_h w_{ho}\phi_h\left(\sum_i w_{ih}x_i\right)\right)$$

The activation functions for the hidden layer, ϕ_h , are almost always logistic. If identity functions are used for the hidden layer and for the output, the resulting NN is quite similar to the partial least squares approach of Wold et al. (1984). We will set one of our inputs to be constant at one so as to allow for an intercept/bias term. The choice of output activation function depends on the nature of the response. For continuous unrestricted output, an identity function is appropriate, while for response bounded between zero and one, such as a binomial proportion, a logistic function should be used. We show the feed-forward NN in Figure 17.3. Sometimes a direct

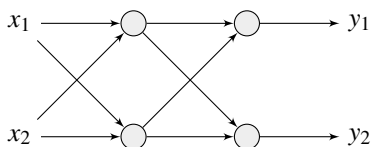


Figure 17.3 *Feed-forward neural network with one hidden layer.*

connection between the inputs and outputs is added called a *skip-layer* connection. More complexity can be added using more layers or feedbacks although this not always beneficial to the practical performance of the NN.

NNs can be elaborated to perform as universal approximators. This has been shown by authors such as Hornik et al. (1989). However, these results are of little practical value when confronted with a finite amount of data subject to noise. We must use the data to estimate the parameters of the model or, in NN-speak, use the data to train the network. The weights w are chosen to minimize a criterion, such as $MSE = \sum (y - \hat{y})^2$ where y is the observed output and $\hat{y}$ is the predicted output. A different criterion would be more suitable for categorical responses.

NN researchers have developed different methods of estimation motivated by brain models of learning. These methods have generally not compared well with the numerical analysis-based approaches which are generally faster and more reliable. Nevertheless, in all but the most simple NNs, the criterion is a complicated function of the parameters. The function often has many local minima making it difficult to find the true minimum. A statistician, with no pretensions to mimicking brain functions, would be inclined to use standard methods of numerical analysis such as quasi-Newton methods, conjugate gradients or simulated annealing. The `nnnet` function in R uses the BFGS method, as described in Fletcher (1987).

17.3 NN Application

We apply the NN method to the `ozone` data analyzed in previous chapters. The *nnet* package, due to Venables and Ripley (2002), must be loaded first:

```
library(nnet)
data(ozone, package="faraway")
```

We start with just three variables for simplicity of exposition as in previous analyses. We fit a feed-forward NN with one hidden layer containing two units with a linear output unit:

```
set.seed(123)
nnmdl <- nnet(O3 ~ temp + ibh + ibt, ozone, size=2, linout=T)
# weights: 11
initial value 65447.874069
final value 21115.406061
converged
```

If you repeat this, your result may differ slightly because of the random starting point of the algorithm, but you will likely get a similar result. We set the random number seed for reproducibility.

The RSS of 21,115 is equal to $\sum_i (y_i - \bar{y})^2$, so the fit is not any better than the null model. The problem lies with the initial selection of weights. It is hard to do this well when the variables have very different scales. The solution is to rescale the data to have zero mean and unit variance:

```
sx <- scale(ozone)
```

Because a random starting point is used, the algorithm will not necessarily converge to the same solution if the fitting is repeated. Now we try refitting the model. We repeat this 100 times because of the random starting point. Here we find the best fit of the 100 attempts:

```
bestrss <- 10000
for(i in 1:100){
  nnmdl <- nnet(O3 ~ temp + ibh + ibt, sx, size=2, linout=T, trace=F)
  cat(i, nnmdl$value, "\n")
  if(nnmdl$value < bestrss){
    bestnn <- nnmdl
    bestrss <- nnmdl$value
  }
}
bestnn$value
[1] 88.031
```

The criterion function has 11 parameters or weights and has multiple minima. The problem is that we can never really know whether we have found the true minimum. All we can do is keep trying and stop if we do not find anything better after some number of attempts. The best strategy is not clear, although one can do better than the simple approach we have used above. Examine the estimated weights:

```
summary(bestnn)
```

```
a 3-2-1 network with 11 weights
options were - linear output units
b->h1 i1->h1 i2->h1 i3->h1
  1.12 -0.98  0.84  0.29
b->h2 i1->h2 i2->h2 i3->h2
137.89 -74.74 240.66 137.89
b->o  h1->o  h2->o
  2.59 -4.41  0.67
```

The notation $i2 \rightarrow h1$, for example, refers to the link between the second input variable and the first hidden neuron. b refers to the bias, which takes a constant value of one. We see that there is one skip-layer connection, $b \rightarrow o$, from the bias to the output.

NNs have some drawbacks relative to competing statistical models. The parameters of an NN are uninterpretable whereas they often have some meaning in statistical models. Furthermore, NNs are not based on a probability model that expresses the structure and variation. As a consequence, there are no standard errors. It is possible to graft some statistical inference onto this NN model, but it is not easy. The bootstrap is a possible way of implementing this. The R^2 for the fit is:

```
1-88.03/sum((sx[,1]-mean(sx[,1]))^2)
```

```
[1] 0.73243
```

which is very similar to the additive model fit for these predictors.

Although the NN weights may be difficult to interpret, we can get some sense of the effect of the predictors by observing the marginal effect of changes in one or more predictor as other predictors are held fixed. Here, we vary each predictor individually while keeping the other predictors fixed at their mean values. Because the data has been centered and scaled for the NN fitting, we need to restore the original scales. The fits are shown in Figure 17.4:

```
ozmeans <- colMeans(ozone)
ozscales <- apply(ozone,2,sd)
xx <- expand.grid(temp=seq(-3,3,0.1),ibh=0,ibt=0)
plot(xx$temp*ozscales['temp']+ozmeans['temp'], predict(bestnn,new=xx)*
     ↪ ozscales['O3']+ozmeans['O3'], xlab="Temp", ylab="O3", type="l")
xx <- expand.grid(temp=0,ibh=seq(-3,3,0.1),ibt=0)
plot(xx$ibh*ozscales['ibh']+ozmeans['ibh'], predict(bestnn,new=xx)*
     ↪ ozscales['O3']+ozmeans['O3'], xlab="IBH", ylab="O3", type="l")
xx <- expand.grid(temp=0,ibh=0,ibt=seq(-3,3,0.1))
plot(xx$ibt*ozscales['ibt']+ozmeans['ibt'], predict(bestnn,new=xx)*
     ↪ ozscales['O3']+ozmeans['O3'], xlab="IBT", ylab="O3", type="l")
```

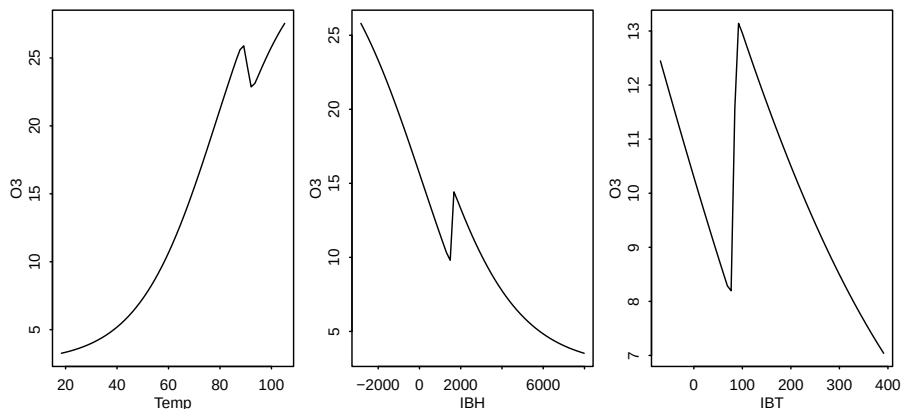


Figure 17.4 *Marginal effects of predictors for the NN fit. Other predictors are held fixed at their mean values.*

We see some surprising discontinuities in the plots which do not seem consistent with what we might expect for the effect of these predictors. If we examine the weights

for this NN above, we see several large values. Consider that all the variables have been scaled to mean zero and variance one. Products formed using the large weights will vary substantially. The situation is analogous to the collinearity problem in linear regression where unreasonably large regression coefficients are often seen. The NN is choosing extreme weights in order to optimize the fit, but the predictions will be unstable, especially for extrapolations.

We can use a penalty function, as with smoothing splines, to obtain a more stable fit. Instead of minimizing *MSE*, we minimize:

$$MSE + \lambda \sum_i w_i^2$$

In NN terms, this is known as *weight decay*. The idea is similar to ridge regression. Let's try $\lambda = 0.001$ for 100 NN model fits:

```
bestrss <- 10000
for(i in 1:100){
  nnmdl <- nnet(O3 ~ temp + ibh + ibt, sx, size=2, linout=T, decay
    ↪ =0.001, trace=F)
  cat(i, nnmdl$value, "\n")
  if(nnmdl$value < bestrss){
    bestnn <- nnmdl
    bestrss <- nnmdl$value
  }
}
bestnn$value
[1] 92.055
```

The value of the best RSS is somewhat larger than before. We expect this since weight decay sacrifices some fit to the current data to obtain a more stable result. We repeat the assessment of the marginal effects as before and display the results in Figure 17.5:

```
xx <- expand.grid(temp=seq(-3,3,0.1), ibh=0, ibt=0)
plot(xx$temp*ozscales['temp'] + ozmeans['temp'], predict(bestnn, new=xx) *
  ↪ ozscales['O3'] + ozmeans['O3'], xlab="Temp", ylab="O3", type="l")
xx <- expand.grid(temp=0, ibh=seq(-3,3,0.1), ibt=0)
plot(xx$ibh*ozscales['ibh'] + ozmeans['ibh'], predict(bestnn, new=xx) *
  ↪ ozscales['O3'] + ozmeans['O3'], xlab="IBH", ylab="O3", type="l")
xx <- expand.grid(temp=0, ibh=0, ibt=seq(-3,3,0.1))
plot(xx$ibt*ozscales['ibt'] + ozmeans['ibt'], predict(bestnn, new=xx) *
  ↪ ozscales['O3'] + ozmeans['O3'], xlab="IBT", ylab="O3", type="l")
```

We see that the fits are now plausibly smooth. Note that *ibh* is strictly positive in practice so the strange behavior for negative values is irrelevant. Compare these plots to Figure 15.3. The shapes are similar for temperature and *ibh*. The *ibt* plot looks quite different although we have no way to assess the significance of any of the terms in the NN fit.

NNs have interactions built in so one should also look at these. We could produce analogous plots to those in Figure 17.5 by varying two predictors at a time.

Now let's look at the full dataset. We use four hidden units because there are now more inputs.

```
bestrss <- 10000
for(i in 1:100){
  nnmdl <- nnet(O3 ~ ., sx, size=4, linout=T, trace=F)
  cat(i, nnmdl$value, "\n")
}
```

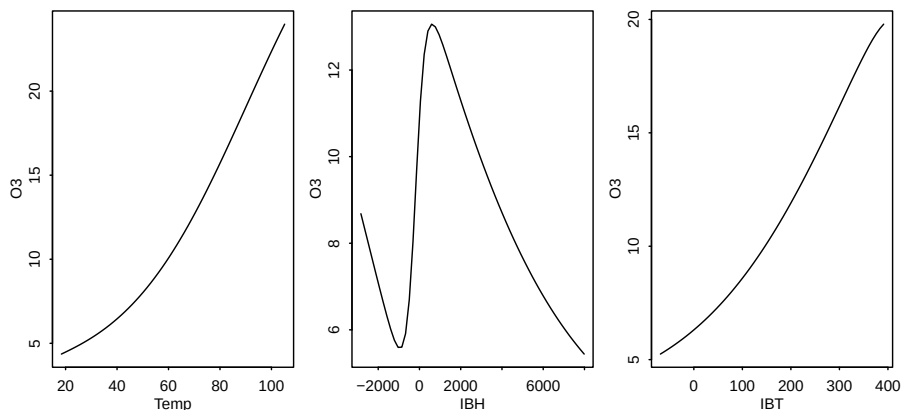


Figure 17.5 *Marginal effects of predictors for the NN fit with weight decay. Other predictors are held fixed at their mean values.*

```
if(nnmdl$value < bestrss){
  bestnn <- nnmdl
  bestrss <- nnmdl$value
}
1-bestnn$value/sum((sx[,1]-mean(sx[,1]))^2)
[1] 0.85063
```

The fit is good and there may be better minimum than we have found and increasing the number of hidden units would always improve the fit. The fit can be compared to those in previous chapters. The R^2 s for the linear, tree and MARS model fits were not as impressive but these approaches tell us more about the relationship between the predictors and the response, so it would not be sensible to judge by R^2 alone. It would be rash to draw firm conclusions from just one dataset. Furthermore, the value of the modeling approaches needs to be judged within the context of the particular problem. If explanation is the main goal of the data analysis, NNs are not a good choice. If prediction is the objective, we cannot judge just by the fit to the data we have now. It is more important how the model performs on future observations. We do not have fresh data here as we have used it all to fit the data. It is a good idea to withhold data for use in testing the prediction performance of the models considered. NNs have been generally competitive in comparison studies but by no means dominant.

17.4 Conclusion

NNs, as presented here, are a controlled flexible class of nonlinear regression models. By adding more hidden units we can control the complexity of the model in a measured way from relatively simple models up to models suitable for large datasets with complex structure. NNs are also attractive because they require less expertise to use successfully compared to statistical models. Nevertheless the user must still pay attention to basic statistical issues involving transformation and scaling of the data

and outliers and influential points. See Faraway and Chatfield (1998) for an example of the application of neural networks and how they compare with statistical methods.

NNs are generally good for prediction but bad for understanding. The NN weights are almost uninterpretable. Although one can gain some insight from plotting the marginal effect of predictors, the NN inevitably introduces complex interactions that often do not reflect reality. Furthermore, without careful control, the NN can easily overfit the data resulting in overoptimistic predictions.

NNs are quite effective for large complex datasets compared to statistical methods where the burden of developing an appropriate sampling model can sometimes slow or even block progress. NNs do lack good statistical theory for inference, diagnostics and model selection. Of course, they were not developed with these statistical considerations in mind, but experience shows that such issues are often important.

NNs can outperform their statistical competitors for some problems provided they are carefully used. However, one should not be fooled by the evocative name, as NNs are just another tool in the box.

Further Reading: See Ripley (1996), Bishop (1995), Haykin (1998), Neal (1996) and Hertz et al. (1991) for more on NNs.

Exercises

1. Consider the `diabetes` dataset analyzed in the exercises for Chapter 16. Before starting, plot the data and construct a reduced dataset without missing values as described in that question.
 - (a) Compute the total sum of squares for the response, `glyhb`. This is the sum of squares about the mean. Now fit a neural network model with this response and the remaining variables as predictors. Use two hidden nodes. Do not rescale and fit the model only once. Compute the residual sum of squares. How has the model performed?
 - (b) Rescale all the numerical variables in the data frame and recompute the total sum of squares. Refit the same neural network model but to the rescaled data frame. Has the model done any better?
 - (c) Fit the model of the previous question 100 times, saving the RSS on each iteration. Plot the RSS against the iteration index. Comment. Do you think more iterations would help?
 - (d) The number of steps within each model fit is limited to 100 and is controlled by the argument `maxit` in the `nnet` call. Set this argument to 200 and repeat the model fitting 100 times as in the previous question. Does increasing this argument improve the fit of the models?
 - (e) Fit the model 100 times as before, but this time save the model producing the best fit. Plot the weights of the best fitting model against the weights from the 100th model fitted. Comment on the relationship.
 - (f) Fit the model 100 times but now use four hidden units. Does this produce a better fit?

2. Refer to the `pima` dataset described in Question 3 of Chapter 15.

- (a) Find all the zero predictor values which are clearly impossible and replace them with missing values. Now fit a neural network model with the diabetes test outcome as the response and the other variables as predictors. Use two hidden units. Do not rescale the data yet. Examine the fitted values from this fit. How do they compare to the proportion of negative and positive test results in the data? How many fitted values are calculated? How does this compare to the number of rows in the data frame?
- (b) Rescale all the predictors to standard units and refit the model. Are the fitted values more informative now?
- (c) Fit the model 100 times and save the best fitting result. What proportion of response values are correctly classified by this best model?
- (d) Construct a test set by taking a random sample of 100 cases from the subset of rows which contain no missing values. Use all the remaining cases as the training set. Fit the model 100 times to the training set and choose the best fit. Evaluate this best model by its performance on the training set. Compare the classification rate achieved with that of the previous question. Which rate is a more realistic representation of how the model might perform on fresh data?
- (e) Increase the number of hidden units to eight. Evaluate the model performance on the full data (fitting 100 times and picking the best model). Now evaluate performance by fitting the models on the training set and classifying the test set. Compare the two performances and comment. Does adding more hidden units help the classification?

3. In this question we model the `Volume` in terms of `Girth` and `Height` using the `trees` data.

- (a) Plot the response on each of the predictors and comment on the relationship. Do they look linear?
- (b) Fit a linear model with `Volume` as the response and the other two variables as predictors. Produce effect displays like those seen in Figure 15.2 for both predictors. Comment on the relationships seen.
- (c) Rescale all the variables to standard units. Fit a neural network model with two hidden units. Repeat 100 times and pick the best fit. Compute the R^2 for this model and compare to the linear model fit.
- (d) Construct a data frame where the (standardized) height varies in the range $[-2, 2]$ and the girth is fixed at the standardized mean of zero. Predict the response for the cases in this data frame. Plot the volume against the height and show your predicted responses as a line on top of this plot. Comment on the shape of the curve. Repeat the exercise, reversing the roles of height and girth.
- (e) Construct a perspective plot showing the bivariate view of the fit. Look at Figure 14.14 for hints on the construction.

This page intentionally left blank